

Security

1 Token stored in localStorage

SafeStore writes the GitHub PAT to localStorage when "remember" is checked. Any XSS on the page (or a browser extension) can steal it. Consider sessionStorage as a safer default, or warn users more explicitly.

2 No Content-Security-Policy meta tag

The page has no CSP header or <meta> tag. This leaves it open to injected scripts if any innerHTML path is ever exploited.

3 innerHTML used with partially-escaped data

Several places build HTML strings with esc() but mix in raw values. For example in DeploymentView.showRun() and ConnectView.updateRemote() — if esc() ever misses a field, it's an XSS vector. UsingtextContent + DOM construction is safer.

4 File MIME type not verified server-side

file.type is checked, but it's browser-supplied and easily spoofed. The extension check is the real gate, which is fine — but worth noting that a .jpg file containing malicious content could be uploaded.

5 No sandbox attribute restrictions on the preview <iframe>

The iframe has sandbox="" (fully restricted) — this is actually  good. But it means CSS from the host page won't load, which is expected but could confuse users.

Robustness / Logic

6 No AbortController on fetch requests

If a user clicks a file, then quickly clicks another, both requests race. The second response could arrive first and get overwritten by the first. There's no cancellation mechanism.

7 No fetch timeout

Requests to api.github.com can hang indefinitely. A slow network will leave the UI stuck in "loading..." with no

recovery path.

8 `tryRecoverStaleShaCommit` **can silently swallow errors**
In `saveFile()`, the recovery path catches 409/422 and calls `tryRecoverStaleShaCommit`. If that throws, it's caught in an inner `try/catch` that only shows a toast — the outer `finally` still runs, but the user may not understand what happened.

9 `normalizeName` **regex allows .html only — but `create()` doesn't re-validate after `normalizeName`**
Minor: `normalizeName` throws on bad names, but `createPartial()` calls `normalizeName` and then checks `this.state.files.some(...)` — the duplicate check uses the normalized name, which is correct, but the flow could be cleaner.

10 `validateProjectConfig` **path traversal check is incomplete**
It checks for `..` in paths but doesn't check for absolute paths (e.g., `/etc/passwd`). A user could set `partialsDir` to `/some/absolute/path` and the API call would likely 404, but it's not explicitly blocked.

Performance / UX

11 **No debounce on the editor input event**
`refreshDirtyUI()` is called on every single keystroke. It re-renders the entire file list (`renderFileList()`) on every keypress. For large file lists this could cause noticeable jank.

12 `loadMediaList()` **is called twice after upload**
In `uploadMediaFiles()`, after the loop it calls `await this.loadMediaList()` and then sets a `setTimeout` to call it again 1.2 seconds later. This causes two redundant API calls.

13 **No offline/network state detection**
There's no `navigator.onLine` check or `offline` event listener. If the user loses connectivity mid-edit, they'll only find out when a commit fails.

14 `refreshAll()` **reloads config + list but doesn't reopen the**

current file

If a file is open and you hit "refresh", the file list reloads but the open file's SHA is now stale. The next commit will likely hit a conflict.

Accessibility

15 Status bar (`#t-status`) has no `aria-live`

The deploy panel has `aria-live="polite"`, but the main status bar at the bottom doesn't. Screen reader users won't hear status updates like "committed" or "conflict detected".

16 Conflict banner uses `<div>` with no `role="alert"`

The `#conflict` element appears dynamically and is important — it should have `role="alert"` or `aria-live="assertive"` so screen readers announce it immediately.

17 Toast notifications have no `role="status"` or `aria-live`

Toasts appear and disappear in 4 seconds with no ARIA announcement.

Minor / Code Quality

18 `_error` and `_networkError` suppressed silently

Several `catch(_error){}` blocks swallow errors entirely (e.g., in `SafeStore`, `dec64` fallback). This makes debugging harder — at minimum a `console.warn` would help.

19 `enc64` and `arrayBufferToBase64` are near-duplicates

Both do chunked base64 encoding. They could be unified into one utility.

20 `defaultProjectConfig()` is called in multiple places instead of being a constant

It returns a new object each time, which is fine functionally, but a `Object.freeze`'d constant would be cleaner and prevent accidental mutation.

Summary table:

Severity	Count
 Security	4
 Robustness/Logic	5
 Performance/UX	4
 Accessibility	3
 Code Quality	4

The most important ones to fix are **#1 (localStorage token)**, **#6 (no AbortController)**, **#7 (no fetch timeout)**, and **#11 (no debounce)**. Want me to suggest fixes for any of these?